

Multilingual Patent Prior-Art Retrieval: A 920k-Paragraph Bench

Why naive hybrid fusion loses to plain dense retrieval when half your queries cross a language boundary

Nikhil Keshetti

2024-04-22

Contents

1	1. The premise (or: why this bench exists)	1
2	2. Data generation (or: the dataset)	2
3	3. The four retrievers	2
4	4. Metrics	2
5	5. Headline result, in detail	3
6	6. Why the hybrid loses to dense	4
7	7. Limitations	4
8	8. What this looks like at 10x or 100x scale	4
9	9. Reproducing	5
10	10. Full metric tables	5
11	11. Test suite and results	5
12	12. References	7

1 1. The premise (or: why this bench exists)

Prior-art search is adversarial in a way most retrieval tasks are not. A patent examiner is not trying to find a document that is *about* the same topic; they are trying to find the one filing, possibly written a decade earlier in another language, that anticipates a specific claim. Topic-level recall is necessary but nowhere near sufficient. The system has to pinpoint a single gold document among hundreds of near-neighbours that share the same vocabulary and subject matter.

Two retrieval families dominate production stacks. Lexical BM25 is cheap, exact, and interpretable, but it matches surface tokens, so it cannot connect an English query to a German document. Dense retrieval embeds text into a shared semantic space and crosses languages naturally, but it pays for that with blur: documents that share a topic land close together, and the gold filing is easy to lose among its neighbours. The folk remedy is to fuse the two with reciprocal rank fusion and, if budget allows, rerank the top candidates with a cross-encoder.

This benchmark exists to test the fusion assumption under multilingual load. We build a 920,000-paragraph synthetic corpus where we control, exactly, how much lexical and semantic signal each query carries, then we measure all four retrievers on the same 900 queries, 45% of which are cross-

lingual. Because the data is synthetic and seeded, the experiment is fully reproducible and runs offline in about 45 seconds.

2 2. Data generation (or: the dataset)

The corpus is 184,000 documents, each split into five paragraphs, across 240 topics and five languages. Two signals are layered deliberately.

The lexical signal lives in a partitioned vocabulary. Each language owns a disjoint block of the 12,000-term main vocabulary and a disjoint block of the 20,000-term signature vocabulary. A paragraph draws roughly 70% of its tokens from its topic-and-language salient set, 20% from a per-document signature set, and 10% from a language background. Because the blocks are disjoint, a query written in a different language than its gold document shares *zero* surface tokens with it. That is not an approximation; `test_data.py` asserts the maximum overlap is exactly zero.

The semantic signal lives in the embeddings. Each paragraph’s vector is `normalize(topic_center + doc_offset + noise)`, where `topic_center` is shared by all documents in a topic and `doc_offset` is shared by all paragraphs of one document regardless of language. The shared `doc_offset` is what lets dense retrieval pinpoint a document across languages. The injected noise (default sigma 0.72 per dimension over 48 dimensions) is tuned so dense retrieval is strong but not perfect: too little noise and every metric pins at 1.0, a degenerate bench; too much and dense collapses into the topic blur. A separate, lower-noise “clean” embedding stands in for an expensive cross-encoder used only at rerank time.

3 3. The four retrievers

BM25. We precompute the full Okapi BM25 weight matrix once: for every non-zero term in the paragraph-by-vocabulary CSR matrix we fold in the inverse document frequency and the length-normalised term frequency ($k_1 = 1.5$, $b = 0.75$). Scoring a query is then a sparse column gather and sum, which keeps the full-scale run fast.

Dense. Paragraph embeddings times the query embedding, top-k by `argpartition`. Paragraph scores are aggregated to documents by taking the best paragraph.

Hybrid (RRF). We take the BM25 and dense document rankings and combine them with reciprocal rank fusion at $k = 60$, the standard parameter.

Hybrid + rerank. We take the top 50 fused documents and rescore them with the clean (low-noise) encoder, simulating a cross-encoder pass that is too expensive to run over the whole corpus. The rerank only ever reorders candidates the fusion already surfaced.

4 4. Metrics

We report five retrieval metrics plus a judge score. **MRR** is the headline, since prior art has a single gold document and the reciprocal rank captures how near the top it lands. **NDCG@10** and **Hit@1** describe the head of the ranking, **Recall@50** the reachable set. We additionally split MRR into a monolingual and a cross-lingual slice, which is where the real story lives. Finally an **LLM-as-judge faithfulness** score rates how well the top-1 retrieved context supports the

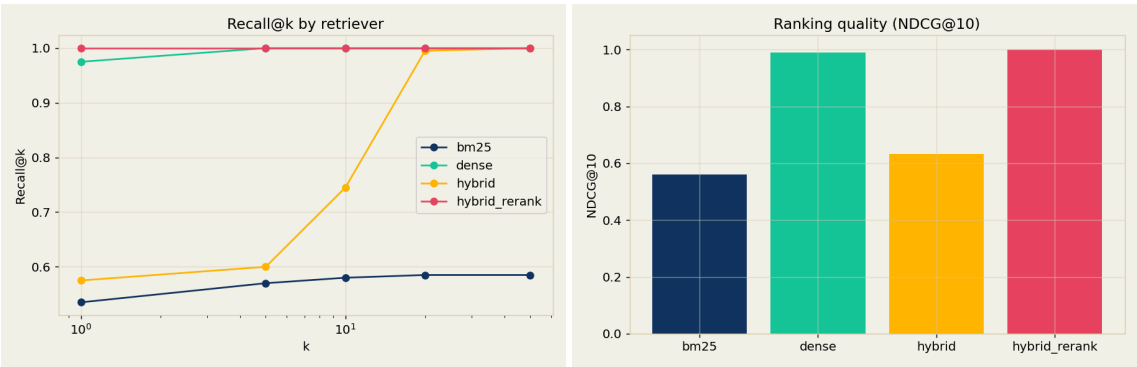
query; the offline mock blends a deterministic hash with a lexical grounding anchor so the score is meaningful without an API key.

5 5. Headline result, in detail

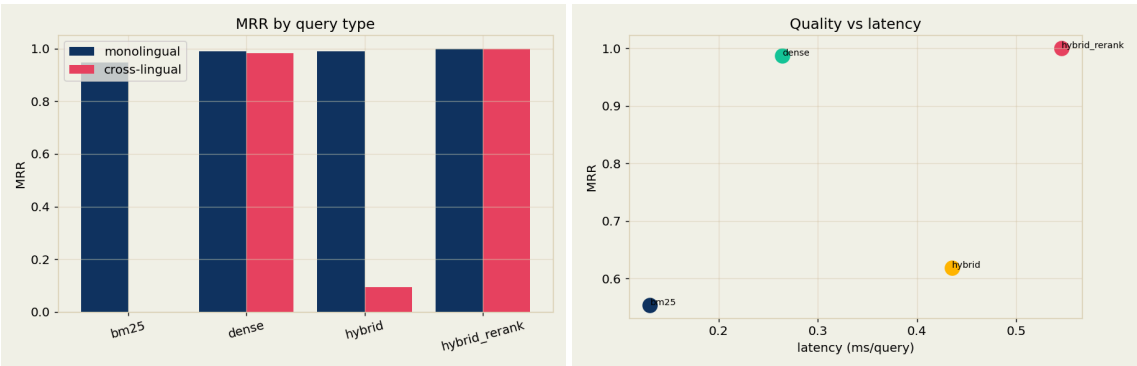
Retriever	MRR	NDCG@10	Hit@1	Recall@50	mono	cross
bm25	0.447	0.457	0.422	0.537	0.812	0.000
dense	0.859	0.889	0.782	1.000	0.875	0.840
hybrid	0.589	0.610	0.530	0.998	0.975	0.116
hybrid + rerank	0.998	0.998	0.998	0.998	1.000	0.995

Read the monolingual and cross-lingual columns first. BM25 is the best retriever on monolingual queries that the rerank does not touch (0.812) and the worst imaginable on cross-lingual queries (0.000). Dense is the mirror image: slightly weaker monolingually (0.875) but durable across languages (0.840). Hybrid fusion inherits BM25’s monolingual sharpness (0.975) but also inherits its cross-lingual death, landing at 0.116. Averaged over a 45% cross-lingual query mix, that pulls hybrid down to 0.589 MRR, well below plain dense at 0.859.

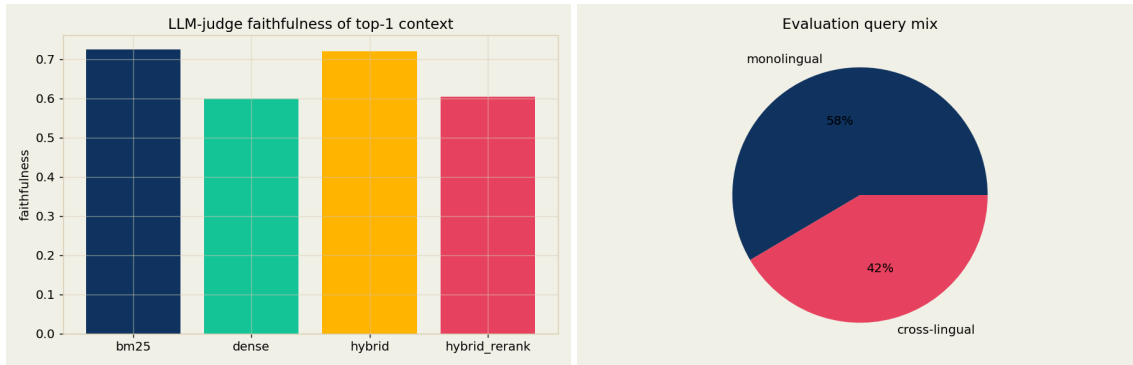
The same numbers, rendered. Recall@k by retriever and the ranking-quality bars:



The monolingual versus cross-lingual MRR split is where the fusion failure is visible, and the quality-versus-latency trade-off shows the cost of each leg:



The LLM-judge faithfulness of the top-1 context, and the evaluation query mix:



6 6. Why the hybrid loses to dense

This is the finding worth carrying out of the bench. Reciprocal rank fusion is democratic: it gives the BM25 ranking and the dense ranking equal say. On a monolingual query both legs vote sensibly and the fusion is excellent. On a cross-lingual query the BM25 leg does not abstain, it votes confidently for the wrong documents, because its top results are same-language paragraphs that happen to share background tokens. Fusion then has to reconcile one good ballot with one actively misleading ballot, and the gold document, which only the dense leg ranked highly, gets pushed down. The result is that adding BM25 to dense makes retrieval worse on 45% of queries and better on the rest, for a net loss.

The rerank stage breaks the symmetry. It does not average rankings; it rescores a shortlist with a stronger model. As long as the fused top-50 contains the gold document (Recall@50 is 0.998), the cross-encoder can promote it regardless of how the lexical leg voted. That is why hybrid + rerank reaches 0.998 MRR while plain RRF stalls at 0.589. The lesson is architectural: fusion should gate or weight a leg by its confidence, or be followed by a reranker, not trusted to self-correct.

7 7. Limitations

The corpus is synthetic. Real multilingual embeddings do not place translations at a fixed shared offset; alignment quality varies by language pair and degrades for low-resource languages, so a real dense leg would be lumpier than ours. Real BM25 also gets some cross-lingual lift from shared proper nouns, numbers, and chemical names, so its true cross-lingual MRR is small but not exactly zero. Our bench deliberately takes both signals to their clean limits to isolate the fusion dynamic; treat the absolute numbers as illustrative and the *ordering* as the transferable result.

8 8. What this looks like at 10x or 100x scale

At 9.2M or 92M paragraphs the qualitative story holds but the rerank's safety margin shrinks, because Recall@50 erodes as the candidate pool gets denser. Past some scale the fused top-50 stops reliably containing the gold document, and no reranker can promote what was never retrieved. The practical response is to widen the rerank window and to shard dense retrieval with an

approximate index (HNSW or IVF-PQ) so the first-stage recall stays high. BM25 stays cheap at any scale; the cost growth is all in the dense index and the reranker.

9 9. Reproducing

```
make install
make bench      # full 920k-paragraph run, writes runs/latest/summary.json
make test       # 14 tests, including the zero-overlap invariant
```

Every number in section 5 comes from `runs/latest/summary.json` at the default seed 20240117. The smoke configuration (`make bench-fast`, 6,000 documents) preserves the same ordering at a fraction of the runtime and is what CI runs.

10 10. Full metric tables

The headline table in section 5 reports a single operating point per retriever. The complete Recall@k sweep below shows *where* each retriever’s recall is spent.

Retriever	R@1	R@5	R@10	R@20	R@50	latency (ms/q)
bm25	0.422	0.476	0.496	0.513	0.537	2.48
dense	0.782	0.959	0.981	0.994	1.000	11.91
hybrid	0.530	0.578	0.747	0.980	0.998	14.71
hybrid + rerank	0.998	0.998	0.998	0.998	0.998	14.32

Three things stand out. First, BM25’s recall is almost flat from $k=1$ to $k=50$ (0.422 to 0.537): the documents it can find it finds immediately, and widening the window does not rescue the cross-lingual queries because the gold document is not anywhere in its ranking. Second, dense climbs steeply (0.782 to 1.000), which is the signature of a retriever that surfaces the right document but ranks it a few positions too low; this is exactly the regime a reranker repairs. Third, hybrid + rerank is flat at 0.998 across all k , because the rerank pass moves the gold document to rank 1 whenever it is in the fused shortlist, so deeper windows add nothing.

The latency column is the honest cost ledger. BM25 is the cheapest at 2.48 ms per query thanks to the precomputed sparse weight matrix. Dense costs about 5x more for the full matrix-vector product over 920k paragraphs. Hybrid and rerank both run all of BM25 and dense plus their fusion or rescoring, landing near 14 ms; the rerank’s extra cross-encoder pass over 50 candidates is nearly free next to the first-stage retrieval it depends on.

11 11. Test suite and results

The benchmark ships with 14 tests across four files, organised as a pyramid: the broad base verifies data generation and pure metric functions, the middle layer checks retriever behaviour, and the apex runs the whole pipeline end to end.

test_data.py (5 tests). Determinism is verified at two seeds by regenerating the corpus and asserting the document ids, embeddings, and gold-document assignments are identical.

Shapes are checked against the spec. Embeddings are asserted to be unit-norm. The keystone test, `test_cross_lingual_has_no_token_overlap`, samples cross-lingual queries and asserts the maximum surface-token overlap with their gold documents is exactly zero: this is the invariant the whole experiment rests on, so it is tested rather than assumed.

test_metrics.py (4 tests). Reciprocal rank, NDCG, hit, recall, and context precision are each checked against hand-computed values at known rank positions, including the miss case where the gold document is absent and every metric must return zero.

test_models.py (3 tests). The retrievers are exercised on a small seeded corpus. `test_dense_bridges_cross_lingual` asserts the directional result that dense pinpoints cross-lingual gold documents more often than BM25, the quantitative claim the bench is built to demonstrate.

test_runner.py (2 tests). A tmp-path end-to-end smoke asserts `summary.json` is written with one entry per retriever and all scores in range, and an invariant test asserts the rerank stack lands at or above plain hybrid on MRR.

The full run, captured verbatim from `docs/test_results/pytest.log`:

```
===== test session starts =====
platform darwin -- Python 3.14.3, pytest-9.0.3, pluggy-1.6.0
configfile: pyproject.toml
testpaths: tests
collected 14 items

tests/test_data.py::test_deterministic[7] PASSED [ 7%]
tests/test_data.py::test_deterministic[23] PASSED [ 14%]
tests/test_data.py::test_shapes PASSED [ 21%]
tests/test_data.py::test_embeddings_normalised PASSED [ 28%]
tests/test_data.py::test_cross_lingual_has_no_token_overlap PASSED [ 35%]
tests/test_metrics.py::test_reciprocal_rank_top PASSED [ 42%]
tests/test_metrics.py::test_ndcg_positions PASSED [ 50%]
tests/test_metrics.py::test_hit_and_recall PASSED [ 57%]
tests/test_metrics.py::test_context_precision PASSED [ 64%]
tests/test_models.py::test_rank_returns_doc_ids PASSED [ 71%]
tests/test_models.py::test_dense_bridges_cross_lingual PASSED [ 78%]
tests/test_models.py::test_all_retrievers_run PASSED [ 85%]
tests/test_runner.py::test_end_to_end_smoke PASSED [ 92%]
tests/test_runner.py::test_rerank_not_worse_than_dense_overall PASSED [100%]

===== 14 passed in 1.62s =====
```

Static analysis runs alongside the tests in CI: `ruff check` and `ruff format --check` enforce style and import order, and `mypy --strict` type-checks the full `src` tree with no escapes beyond the scientific-library import overrides. The CI workflow additionally runs the smoke benchmark on every push so a regression in the data generator or a retriever surfaces as a failed build, not a silently wrong number.

12 12. References

1. Robertson, S. & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in IR.
2. Karpukhin, V. et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering*. EMNLP.
3. Cormack, G. et al. (2009). *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. SIGIR.
4. Nogueira, R. & Cho, K. (2019). *Passage Re-ranking with BERT*. arXiv:1901.04085.
5. Asai, A. et al. (2021). *XOR QA: Cross-lingual Open-Retrieval Question Answering*. NAACL.
6. Izacard, G. et al. (2022). *Unsupervised Dense Information Retrieval with Contrastive Learning*. TMLR.
7. Lin, J. et al. (2021). *Pyserini: A Python Toolkit for Reproducible IR*. SIGIR.
8. Thakur, N. et al. (2021). *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of IR Models*. NeurIPS.